Danmarks Tekniske Universitet

DTU

AC-Magnetometry: Pulse Sequence Comparison (Ramsey vs Hahn Echo vs CPMG)

Scientific computing in quantum information science

Authors

Bendeguz Farkas

August 19, 2026

Purpose of the project

I want to measure a magnetic field that's oscillating at some known or suspected frequency - current flowing through a cable, a vibrating structural element, a heartbeat's magnetic signature, whatever. The signal is usually tiny and buried in environmental magnetic noise (Earth's field fluctuations, nearby electronics, thermal drift). A classical magnetometer (e.g. a simple coil or Hall sensor) picks up everything indiscriminately signal and noise together - so weak signals get lost.

So the goal would be to make the sensor "listen" selectively at the frequency where the signal lives, while filtering the noise everywhere else - the same idea as a lock-in amplifier or a radio tuner, but implemented with a quantum sensor's.

A single quantum sensor (an NV-center spin, a trapped ion, an atomic ensemble) starts in a superposition state and precesses under any magnetic field present. Left alone, it responds to everything down to DC, including all the noise. The trick is inserting a sequence of control pulses (π-pulses) during that precession. Every pulse flips the sign of how the sensor "remembers" the field it has accumulated so far drifting noise field averages itself out over the sequence, while a field oscillating in sync with the pulse spacing keeps reinforcing constructively. [1]
And that's what I want to simulate: the pulse sequence acts as a tunable bandpass filter, and its center frequency is set by how many pulses I pack into the sensing window.

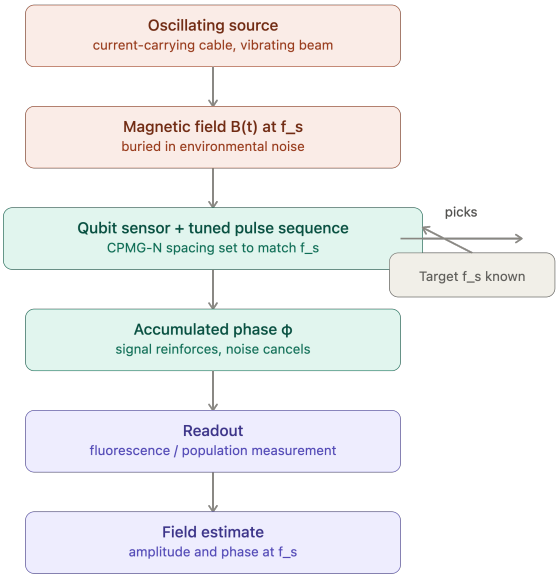

Figure 1: Pipeline of the overall solution

Background

Dynamical decoupling sequences are open-loop quantum control techniques that suppress environmental noise and extend qubit coherence times by applying timed pulses to average out decoherence. These protocols function as filter functions, where the pulse sequence design determines which frequency bands of noise are suppressed.

Each dynamical-decoupling sequence (Ramsey/FID, Hahn echo, CPMG-N) is described by a "modulation function" $y(t)$ in $\{+1, -1\}$. Function $y(t)$ tracks the sign of the qubit's coherence relative to a static reference frame, and it flips sign at every instantaneous π-pulse. This causes positive and negative phase errors to cancel out, effectively acting as a *high-pass filter* that blocks low-frequency environmental fluctuations while allowing the qubit to maintain coherence.

Physical picture looks like that the qubit accumulates phase $\phi = \gamma * \int_0^T y(t) * B(t)dt$ under any magnetic field $B(t)$ (noise + signal). Therefore $y(t)$ is what makes different pulse sequences sensitive to different noise/signal frequencies. [4]

Sequence	**Description**	**Noise Suppression Profile**	**Key Characteristic**
Ramsey/FID	Free Induction Decay, no refocusing pulses	None; sensitive to all low-frequency noise	Baseline coherence time (T_2^*); phase accumulates randomly
Hahn Echo	Single π-pulse at midpoint of evolution	Cancels static fields and low-frequency noise	Reverses phase accumulation; effective for slowly varying noise
CPMG-N	Train of N equally spaced π-pulses	Filters out low-frequency noise, shifts passband higher with N	Extends T_2 significantly; effective against $1/f$ noise (CPMG-2 equals Hahn Echo)

Pulse sequences implementation

First thing first, I need to implement the pulse sequences. Evaluate $y(t)$ for a CPMG like sequence with n pulses equally spaced π-pulses over total free-evolution time T. 0 pulses become Ramsey, $y(t) = +1$ always, pulse 1 is the Hahn echo, one π-pulse at $T/2$ and if pulses equal to N then CPMG-N, pulses at $T/(2N)$, $3T/(2N)$, $5T/(2N)$...
This can be found in the ***pulse_ sequences.py*** file.

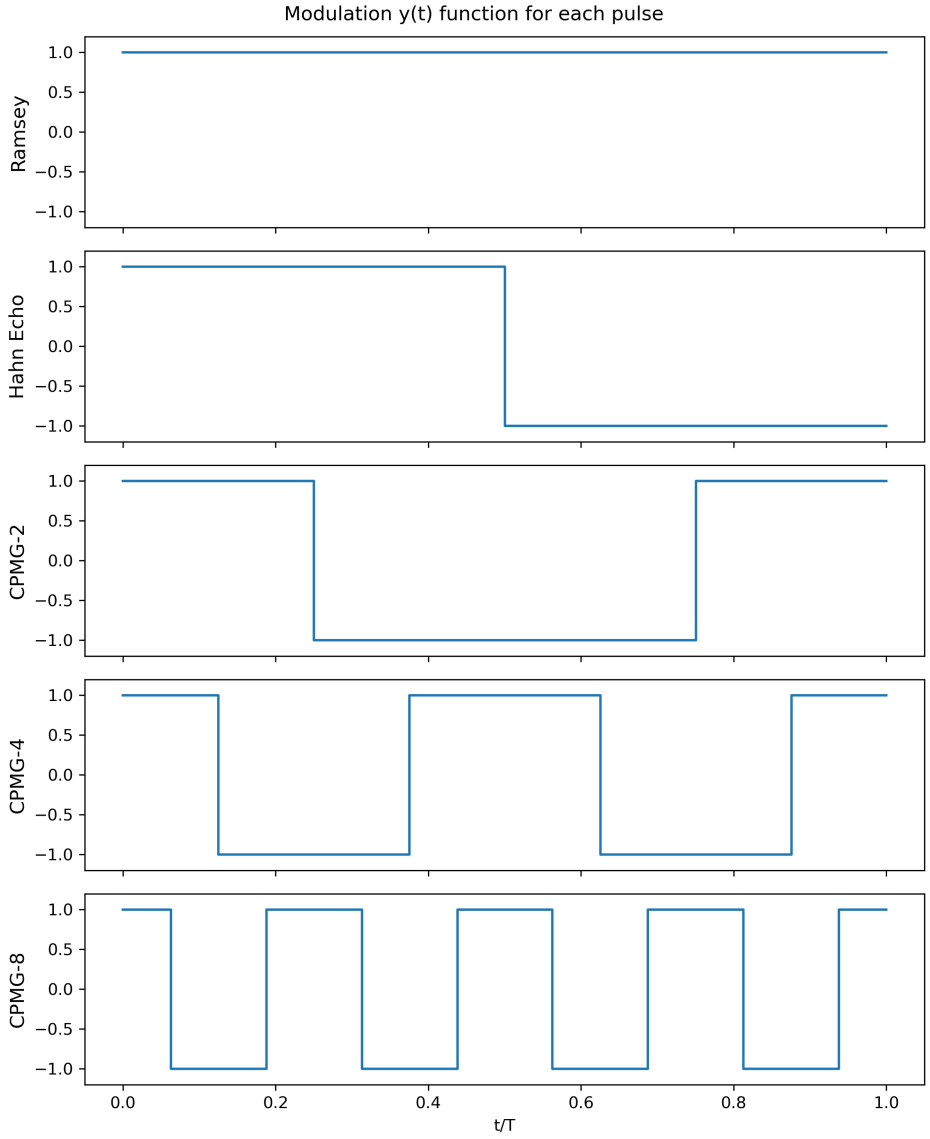

Figure 2: Ramsey stays flat (DC-sensitive), Hahn echo flips once, CPMG-N flips N times.

Filter function

The filter function $F(\omega)$ tells you how sensitive a pulse sequence is to noise/signal at each frequency. It's the (squared) Fourier transform of the modulation function $y(t)$:

$$\chi(\omega) = \int_0^T y(t) e^{i\omega t} dt$$

$$F = |\chi(\omega)|^2$$

This function explains why Ramsey is a DC magnetometer, Hahn echo is a low-pass-rejecting bandpass filter around $\omega \sim \pi/T$, and CPMG-N creates a comb of narrow peaks at odd harmonics of $N * \pi/T$ – which is exactly the "lock-in" behaviour used in real AC magnetometry.[3]
This calculation can be found in ***filter_function.py*** file.

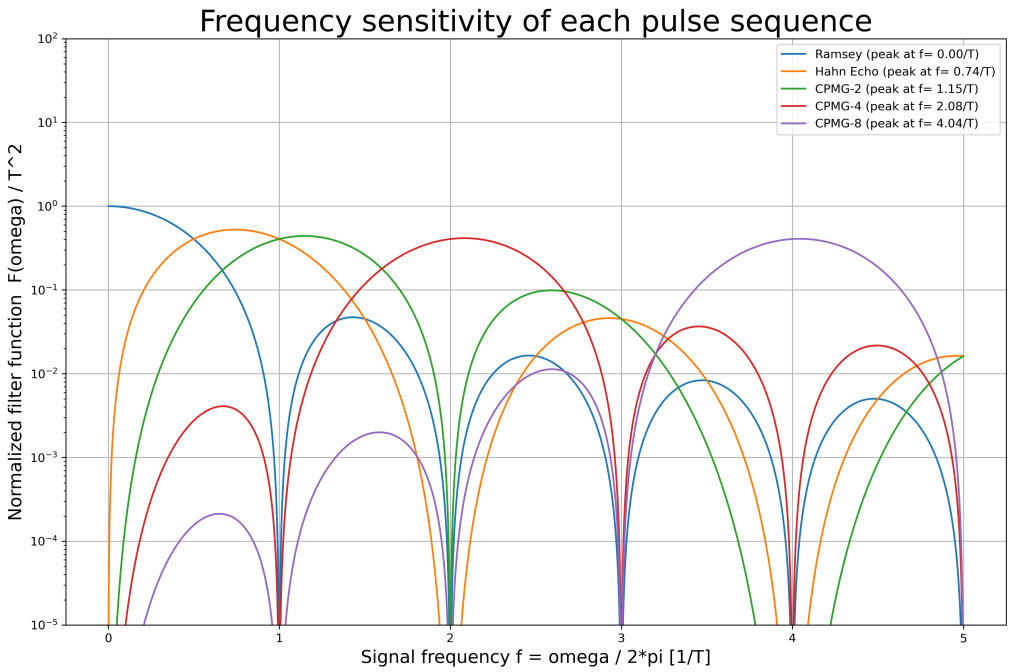

Figure 3: This plot proves the core physics claim: different pulse sequences act as tunable bandpass filters, so we pick our sequence based on what frequency we're trying to detect.

Noise model

Two things happen to a real sensor:

1. Environmental magnetic noise $B(t)$ causes dephasing: this is what limits $T2^*/T2$ and sets our noise floor.
2. The target AC signal $B\_signal(t) = B\_AC * cos(w_s t)$ causes a deterministic phase shift we're trying to detect.

For weak coupling (the standard regime for real magnetometers), both effects enter through the same overlap integral with $y(t)$, so both the noise-induced decay and the signal response are controlled by the filter function $F(\omega)$ computed in ***filter_function.py***:

$$Signal\ response : \phi\_signal(w_s) = \gamma * B\_AC * Re\{\chi(w_s)\}$$

$$Noise\ dephasing : coherence\ decays\ as\ e^{-\chi_2}\ with\ \chi_2 = (\gamma^2/2) * \int d(w)/2\pi * S\_B(w) * F(w)$$

I use a Lorentzian noise power spectral density $S\_B(w)$, which is the standard model for a single dominant noise source with correlation time τ_c.
Predicted coherence magnitude $|<\sigma_+>|/|<\sigma_+>|_0$ after a sensing sequence of total time T, integrating the noise PSD against the filter function (Gaussian approximation).[2][3]
Final function will response to an AC signal of amplitude B_AC at angular frequency ω signal, accumulated over sequence length T and returns a accumulated phase ϕ. For $\phi << 1$ this is directly the measurable signal. This calculations are in the ***noisy_signal.py*** file.

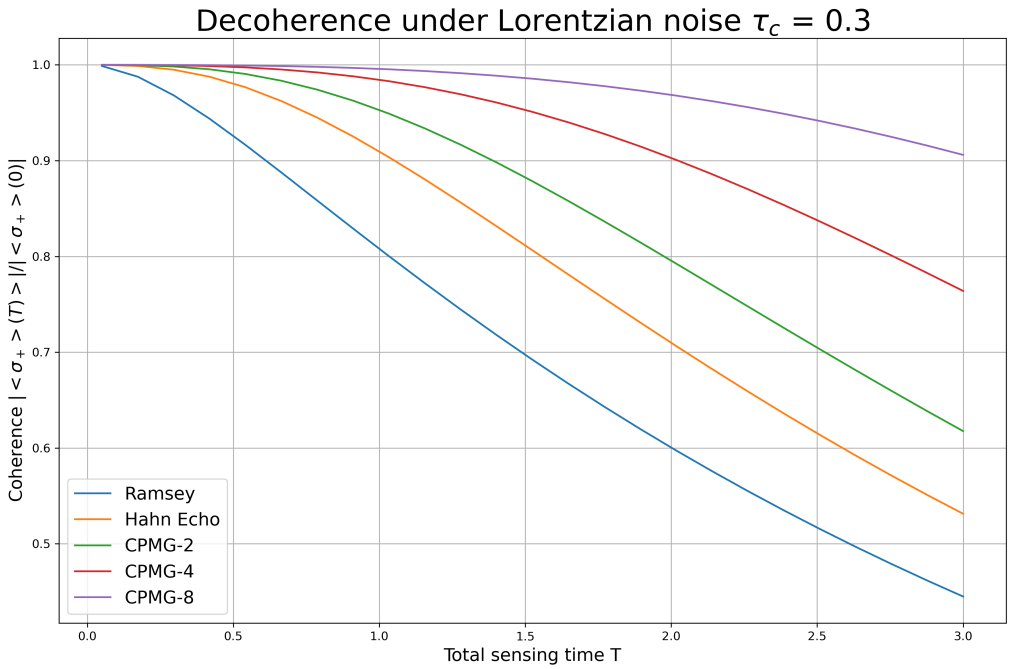

Figure 4: This is the advantage of dynamical decoupling: more pulses protect against slow noise, extending our usable sensing time

Analysis

Now put together what I did so far, I can determine which sequence should I use to detect a signal at specific frequency.

There's a trade-off, what have to consider: with more pulses, we can achive longer usable T_2 coherence time, which means more total phase can accumulate. On the other hand, more pulses also shifts where the sequence is sensitive, which means if our signal is not near to the sequence's peak frequency, than we got nothing.

Sensitivity metric used here [3] for a fixed sensing time T, the achievable signal-to-noise combines first, how strongly the sequence transduces the AC signal into phase, and secondly how much coherence survives environmental dephasing over that T.
We define a sensitivity figure of merit (lower = better):

$$\eta(f_s, T) = 1/(|overlap(f_s, T)| * C_{noise}(T) * \sqrt{T})$$

The $\sqrt{T}$ rewards longer averaging (standard shot-noise scaling); |overlap| rewards resonance with the signal; C_noise penalizes decoherence. This is a simplified but physically faithful stand-in for a real sensitivity calculation.

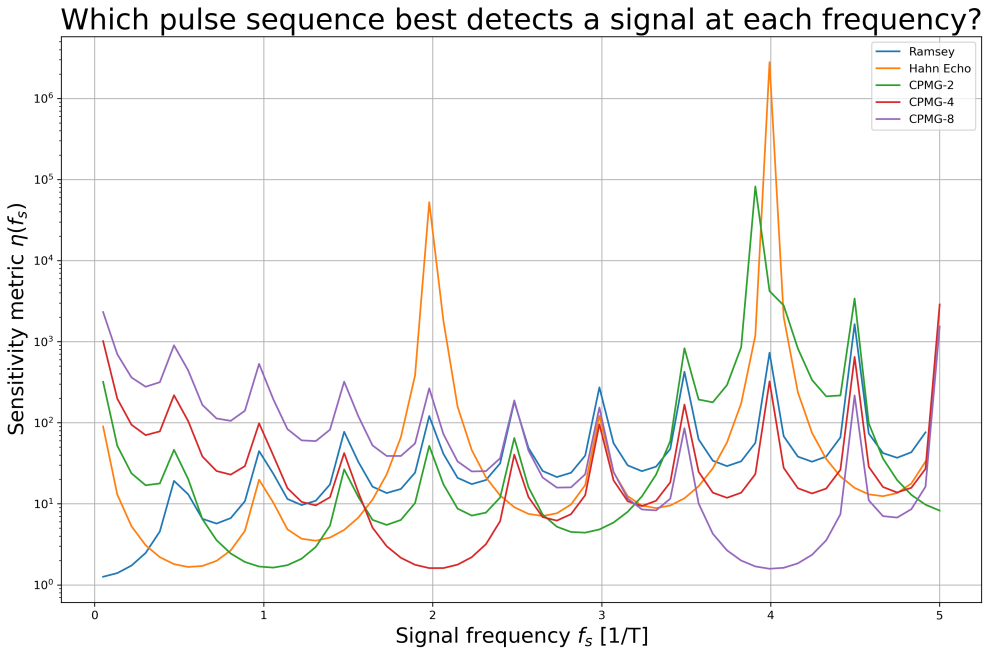

Figure 5: The "best sequence" progresses in order (Ramsey, Hahn, CPMG-2, CPMG-4, CPMG-8) as frequency increases, exactly matching each sequence's filter-function peak location.

Each sequence has its own "sweet spot" (a dip = best sensitivity), and which sequence wins depends entirely on our target frequency. Code in ***analysis.py*** file.

Testing on syntetic lab environment dataset

Now, we can determine which bandpass filter we should use it is time to test our program. I create a noisy dataset to simulate a lab environment and then applying our filter try to extract a 50 Hz signal.
First, our sensing window T sets which frequency each sequence is tuned to (peak near $N/(2T)$). For a 50 Hz target, T = 20 ms with CPMG-2 gives a peak almost exactly at 50 Hz - so that's our natural operating point. I fine-tuned it in the ***visualization.ipynb***, so T = 22.9587 ms with CPMG-2 lands the sequence's peak sensitivity right at 50 Hz. That's what I will use for the whole simulation.
Now the dataset - we simulate the raw magnetic field time series a sensor sitting in a real lab would see: our 50 Hz cable signal plus a handful of realistic interference sources. Sampled finely to resolve everything up to a few hundred Hz.

Noise sources included:

- 50 Hz mains cable (the target signal we want to detect)
- harmonics of the cable itself (100, 150 Hz) – real AC current is rarely a pure sinusoid
- a nearby motor/fan at 23 Hz (mechanical vibration -> magnetic coupling via a ferromagnetic part)
- a second piece of equipment at 62 Hz (switching supply, unrelated device on a different phase)
- slow correlated drift (Lorentzian PSD, e.g. thermal / building baseline wander)
- white readout noise floor

I plot the raw time series (should look like noisy junk) and its power spectrum (should show the 50 Hz target sitting among several other peaks. Now we can see the peaks because we generated the data and know where to look, but its height relative to neighbors, and the broadband noise floor, is what actually limits detection in a real single-shot measurement. That's why we cannot just FFT and read iff 50Hz.

The dataset were created in the ***env_ dataset.py*** and the graphs were made in ***visualization.ipynb*** file.

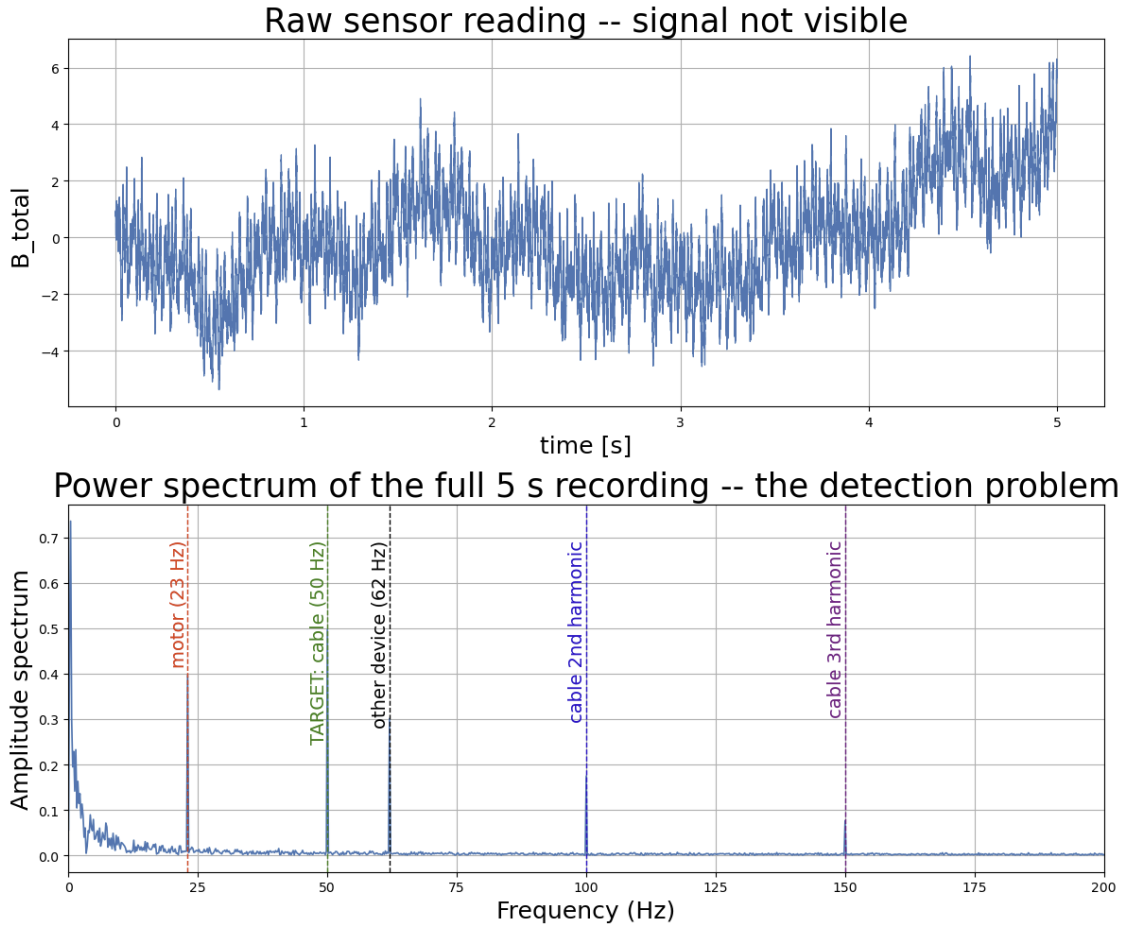

Figure 6: Raw time series on the uppper picture where oscillations are visible and the spectrum of the full 5s recording below that

The final part is to act our sensing program on the dataset. I want to mention that a real quantum sensor doesn't get to FFT a continuous 5s recording like what we saw on plot above. It only gets short T = 22.97 ms windows, and each one ends in a single noisy quantum measurement.
Instead, a sensor runs one pulse sequence over a short window T, accumulating phase $\phi = \gamma * \int y(t)B(t)dt$. It is read out once per shot through projective quantum measurement (a "click" or "no click") - this is the origin of quantum projection noise, an unavoidable source of randomness on top of the real magnetic noise. We can repeat the shot R times to beat down that projection noise, and repeat across many consecutive windows to build up a time series of phase estimates. Due to the target's frequency is known (50 Hz), we can "lock in" the resulting sequence of per-shot phase estimates against a 50 Hz reference.
I slice the dataset into T shot windows to simulate quantum-projection-noise readout for each, and lock-in demodulate the resulting phase-estimate sequence at 50 Hz.

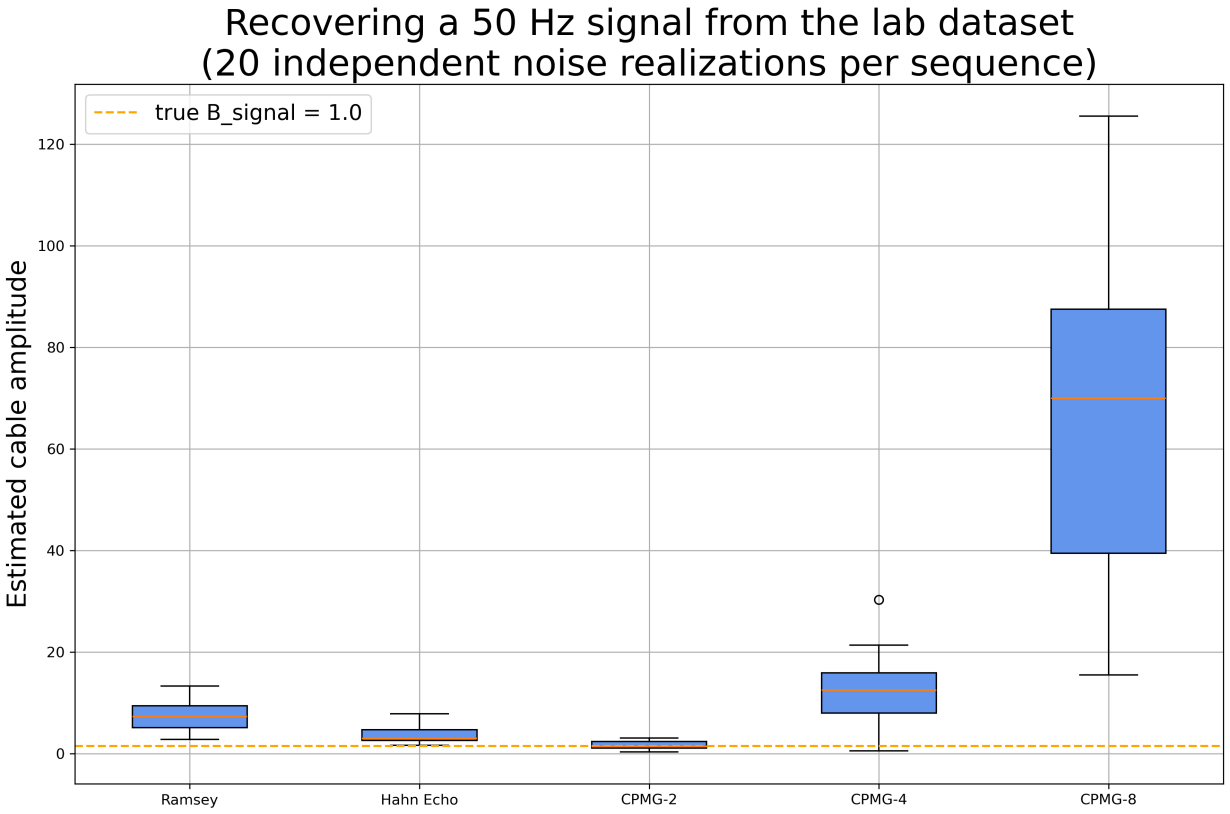

Figure 7: Indeed, CPMG-2 were the closest one to the B signal.

Summary

This project was highly valuable of the perspective that I could understand better and try out dynamic decoupling technique in quantum sensing. However the quantum advantage of this solution is highly depend on the physical implementation what strongly limiting the application starting with the readout challenge (0 or 1) and the projection measurement noise. I also write some possible further development topics in the project's README file. Also want to mention that in software perspective, real NV-center and atomic magnetometer labs use a completely different software stack, purpose-built for pulse-level timing control:

- ARTIQ (Python-based, FPGA-backed, nanosecond timing) - originally built for trapped ions, now widely used for NV centers too
- Zurich Instruments LabOne Q - has NV-center specific application libraries built in
- QCoDeS - Python data-acquisition framework, developed jointly by Copenhagen, Delft, Sydney, and Microsoft Research
- QICK / QubiC - newer open-source FPGA control stacks

References

[1] Anonymous Author(s). *A Quantum Dynamics Tutorial: Visualising Dynamical Decoupling Sequences on the Bloch Sphere*. 2026. arXiv: 2608.10914 [quant-ph].

[2] Łukasz CywiŁski et al. "How to enhance dephasing time in superconducting qubits using dynamical decoupling pulses". In: *Phys. Rev. B* 77 (17 May 2008), p. 174509. DOI: 10.1103/PhysRevB.77.174509. URL: https://link.aps.org/doi/10.1103/PhysRevB.77.174509.

[3] C. L. Degen, F. Reinhard, and P. Cappellaro. "Quantum sensing". In: *Rev. Mod. Phys.* 89 (3 July 2017), p. 035002. DOI: 10.1103/RevModPhys.89.035002. URL: https://link.aps.org/doi/10.1103/RevModPhys.89.035002.

[4] H. Uys. "Dynamical decoupling sequence construction as a filter-design problem". In: (2009). National Laser Centre, Council for Scientific and Industrial Research, Pretoria, South Africa. Email: huys@csir.co.za.